Universidad de La Sabana
Facultad de Ingeniería

Felix Mohr & Sergio Amortegui
2026.1

Fundamentos de Inteligencia Artificial

- Sheet 7-

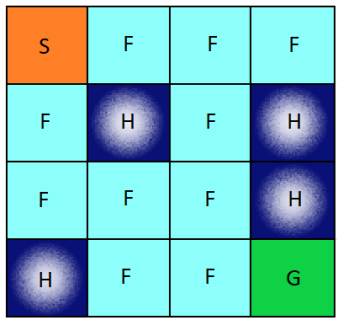
Environment & Rules. Use Python with only `numpy`. Do not use global random state; all randomness must come from a provided `numpy.random.Generator` named `rng`. Unless explicitly stated otherwise, functions that sample must accept a seed or `rng`. Keep the function signatures from the provided `.py` templates unchanged, as they will be tested automatically on Gradescope. You will only see public test outputs.

Files to implement. `bellman.py`, `policy_eval.py`, `my_policy.py`, `run.py` Do not rename files or functions.

Environment Considerations: Lake MDP (reuse). We consider the *Lake* environment on an $m \times n$ grid. Each cell is one of

`S` (unique start), `F` (free), `H` (hole, terminal), `G` (goal, terminal).

At each non-terminal step the agent chooses an action in $\{\text{UP}, \text{RIGHT}, \text{DOWN}, \text{LEFT}\}$. With probability 0.8 the intended move is executed; with probability 0.1 each, the move slips to the two lateral directions if the state is not an absorbing one. Bumping against borders keeps the agent in place.
Rewards depend *only* on the state *entered*: a constant step reward in `F` (-0.35), a fixed reward in `H` (-1.0), and a fixed reward in `G` ($+1.0$). Upon entering any terminal cell, the process transitions to a synthetic absorbing state $s_\perp$ with only action $\perp$ and reward 0 thereafter.



We assume throughout that the discount factor satisfies $\gamma \in (0, 1]$.

Exercise 1 (Exercise 1: Bellman Operator – 1 Points) Complete the code under `bellman.py` that applies a single Bellman update for a fixed policy π. It must compute, for all s,

$$v_{\text{new}}(s) \;=\; r(s) \;+\; \gamma \sum_{s'} P(s' \mid s, \pi(s))\, v(s')\,.$$

Exercise 2 (Exercise 2: Exact Policy Evaluation – 2 Points) Complete the code under `bellman.py` that performs exact policy evaluation by solving the linear system

$$(I - \gamma P)\, v \;=\; r \quad \Longleftrightarrow \quad v \;=\; (I - \gamma P)^{-1} r\,,$$

where P is the policy-induced transition matrix and r the reward vector (consistent with the chosen reward convention). Return the value vector v^π.

Hint: you may find `numpy.linalg.solve` *useful.*

Exercise 3 (Exercise 3: Iterative Policy Evaluation – 2 Points) Complete the code under `policy_eval.py` that computes v^π via successive Bellman updates. Start from an initial estimate (e.g. $v = 0$), repeatedly apply the update

$$v \leftarrow r + \gamma P v$$

and stop when the sup-norm change satisfies the slide criterion

$$\|v_{\text{new}} - v\|_\infty \;<\; \varepsilon \, \frac{1-\gamma}{\gamma} \, .$$

Return the final estimate v.

Exercise 4 (Exercise 4: Policy Construction by Search – 4 Points) Complete `my_policy.py` to implement a constructive search procedure or a generative algorithm that, when instantiated with a new MDP, produces a deterministic policy $\hat{\pi}$.

Some possible design directions include:

1. **Constructive search:** build a policy by following a precomputed potential field or shortest-path/graph heuristic that avoids holes and drives toward the goal (no rewards, no discounted returns).
2. **Generative / transformation methods:** iteratively transform an initial policy using only topology or reachability information (e.g. "point to the neighbor that reduces a distance-to-goal heuristic"), again *not* using value estimates.
3. **Optional GA-style search (value-free):** you may use randomness to propose policy candidates and select among them *only* using surrogate, structure-based objectives (e.g. path length to goal in a slip-free proxy graph, hole-avoidance penalties, reachability), never discounted returns or v^π.

Important: The policy constructor must never compute, observe, or use state-values v^π or returns. Its output is solely the constructed policy $\hat{\pi}$.

Exercise 5 (Exercise 5: Runner with Policy Evaluation – 4 Points) Complete `run.py` to:

1. Construct a policy $\hat{\pi}$ by calling your implementation from Exercise 4 with the provided MDP as input.
2. Evaluate $\hat{\pi}$ using either the exact *or* the iterative policy evaluation method.
3. Define the *fitness for reporting* as the value of the initial state, i.e. fitness $= v^{\hat{\pi}}(s_0)$.
4. Return the triple
$$(\hat{\pi},\ v^{\hat{\pi}},\ f^{\hat{\pi}}),$$
where $\hat{\pi}$ is the constructed policy, $v^{\hat{\pi}}$ the corresponding state-value vector, and $f^{\hat{\pi}}$ the fitness $v^{\hat{\pi}}(s_0)$.

Exercise 6 (Practice: Theory and Sanity Checks) These problems are not graded on Gradescope but will help you understand the concepts and will be discussed in class.

1. Show that the Bellman operator $B : v \mapsto r + \gamma P v$ is a contraction under $\|\cdot\|_\infty$ for $\gamma \in (0, 1]$.
2. Conclude existence and uniqueness of v^π and the convergence of the iterative method to v^π.
3. Derive the linear system in Exercise 2 starting from the Bellman equation.
4. Apply the Bellman update rule to the Lake MDP for an initial estimate $\hat{v}^\pi = 0$ for all states.
5. Give an upper bound on the maximum error in the estimate of the true value of any state after this update.
6. (Slides gridworld) Using a policy as in class, verify numerically that the value at the start state ($S_0 = (1, 1)$) matches the figure from the slides up to small numerical error.
7. Briefly discuss when you would prefer exact vs. iterative policy evaluation in terms of $|S|$ and available memory.